

Module 1 – Introduction & Universal Approximation

Video Transcripts

Video 1.1: Introduction to Neural Networks (09:34)

Welcome to this introductory course on artificial neural networks. Without further ado, let's dive right into the topic. Artificial neural networks have now become the main approaches to AI. They've been successfully applied to various pattern recognition, prediction, and analysis problems, and in many of these, they've established the state of art, often beating benchmarks set using earlier machine learning techniques by large margins. They've even managed to solve some problems that you simply could not solve using earlier methods. Here are some examples of breakthroughs using neural nets. Automatic speech recognition: After over 70 years of research, automatic speech recognition systems still performed terribly until 2015. And then in 2016, a neural network powered automatic speech recognition system outperformed humans on a complex recognition task for the first time.

Automatic machine translation systems were generally not useful until they began using neural networks in 2016. But now, with neural networks, they are so good that even human experts sometimes use them to verify their translation. Here's another impressive example of image analysis with a neural net. This is a rather complicated image, but all the objects in it have been located and tagged with a neural network. Here, for example, the girl on the bicycle, the bicycle, and even the bag she's carrying are all correctly detected and recognized. This level of performance was not possible before neural networks. Similarly, for games like Go. These are what are generally viewed as mental games, requiring human intelligence because of their complexity, and until 2016, no computer had come close to beating a human champion at Go. Then in 2016, AlphaGo, a neural-net based system, beat the world champion.

Today you have programs that don't even need to know the rules and can learn the game from scratch on their own, and in just a few hours become good enough to beat the world champion. This too is thanks to deep neural networks. Here's another challenge. Image captioning: All of these image captions were automatically generated by a neural network: "Man in black shirt is playing a guitar", "Construction worker in orange safety vest working on road", "Boy doing backflip on wakeboard". Generating such captions requires the ability to both interpret the image and generate the language describing it. Such level of sophistication in AI was not possible before neural networks. These results are from 2016. The latest models are much more improved over even this performance.

Here's another example. These are all photographs of people who do not exist. This is a neural network getting creative, simply fantasizing faces from thin air. This kind of creativity was not possible with other types of machine learning models. And many other such problems, ranging from astronomy to healthcare to even stock prediction have been tackled by neural networks. Neural networks are among the best performing models used by investment firms these days,

for example. In fact, most investment firms have created entire research teams devoted to neural networks.

So, what are these amazing neural networks? From where we stand at this point in the course, they are just these magical boxes where things go in and other useful things come out. For example, a voice recording goes in, and out comes a transcription of what was said in the recording. An image goes in, and a caption comes out. A game state goes in, and out comes a recommendation for the next move to make. But what's in these boxes? To understand them, let's begin here. All of these tasks that we just looked at: recognizing speech, captioning images, playing complicated games, these are fundamentally human activities — powered by human intelligence. So, perhaps, the place to begin is with the seat of all human intelligence, the human brain.

The physical structure of the brain was already known by the mid 1800s, thanks to the advances in microscopy at that time. The brain is composed primarily of a very large number of a very specialized type of cell called a neuron, and neurons are all interconnected. Each neuron has many other neurons connecting into it, and each neuron connects out to other neurons. The first person to realize how the brain worked was Alexander Bain, back in the 1870s. Like the scientists of his day, Bain was a bit of everything — he was a philosopher, he was a psychologist, a mathematician, he was a logician, linguist, and he was a professor. Back in the 1800s being a professor was a rather bigger deal than it is today. In his 1873 book, "Mind and Body", Bain postulated that the brain stores information through its connections. All information about the world, or more generally between stimulus and responses, was stored through the structure of the connections between the neurons. This was, in fact, the first connectionist model. While this was controversial in the 19th century, by now, Bain's model is accepted as an established fact.

Our brain is a connectionist machine, as predicted by Bain in 1873 and Ferrier in 1876. Neurons connect to other neurons, and the processing and information capacity of the brain is a function of these connections. This is what a connectionist machine is: it is a network of simple processing elements. All world's knowledge about the task that the machine performs is stored in the connections between the elements, and the complexity of the machine itself is achieved through the complexity of these connections. And modern neural networks emulate the structure of the brain. They are connectionist machines, consisting of a network of many simple processing units and the task performed by them depends on the structure of the network. This is in complete contrast to conventional computers, that use variants of the Harvard or Princeton architecture. In these architectures the computer has a distinct processing unit and a separate memory where programs and data are stored.

If you want to run a new program, all you have to do is to upload a new program into the memory and you're set to go. That is not the case for connectionist machines. For connectionist machines, the computer is the program, because the program and its memory are all encoded in the connections. To change the program, you have to change the machine itself. That's why we usually emulate connectionist machines on regular Harvard and Princeton

architecture machines. So, to recap the story so far: Neural network-based AI has taken over most AI tasks. Neural networks originally began as computational models of the brain. The brain is a connectionist machine: neurons connect to neurons, and the workings of the brain are encoded in these connections. So, current neural network models are connectionist machines.

Video 1.2: The Perceptron (08:22)

Let's pick up where we left off. To recap, neural network-based AI has taken over most AI tasks. Neural networks originally began as computational models of the brain, which is connectionist in nature — information is primarily captured in the connections between the neurons. So, neural network models are connectionist machines.

Connectionist machines are networks of simple processing elements. All the information the machine uses to perform its computations are stored in its connections. But what are the individual processing elements themselves? To understand this, let us return to the brain. The individual units in the brain are neurons. A neuron looks roughly like this: it has a head, called the soma. The soma has several dendrites, through which it receives signals from other neurons. The neuron generates spike trains, the rate of which depends on the total input signal. At a high level, we can view it this way: if the total input from other neurons exceeds the threshold, the neuron fires. This output signal travels down this long leg, called the axon, from which it is transmitted to other neurons.

Here's an interesting factoid — the axon is very long and needs to be protected for the neuron to work, so it is covered with a sheath of fat, called the myelin sheath. This fat is crucial for the functioning of the brain. In fact, scientists have found that the key distinction between Einstein's brain and normal brains was that he had more myelin — than normal people. It appears that the more fat you have in your head, the smarter you are. So, the next time someone calls you a fathead, thank them for the compliment. Anyway, for our computational connectionist models too, we use compute units that emulate the operations of biological neurons.

The computation model of the neuron is the perceptron, first proposed by Frank Rosenblatt in 1958. This figure here represents a perceptron. It receives a number of inputs X_1 through X_N . And corresponding to each input is a weight — shown by the symbols W_1 through W_N in this figure. The weights can be positive or negative. The perceptron computes a weighted sum of these inputs. If the weighted sum exceeds sum threshold T , the perceptron outputs of 1, otherwise it outputs 0. In the form of an equation, we can write it as shown here. If the weighted sum minus the threshold T is greater than or equal to 0, the output goes to 1, otherwise it becomes 0. In spite of its simplicity, the perceptron can mimic any Boolean gate. Consider these examples. Each figure here represents a perceptron. In the figures, the number written on top of each arrow shows the weight of a connection. The number within the circle represents the threshold. In the top left figure, if both inputs X and Y are 1, then their weighted sum is 2 and matches the threshold, so the perceptron outputs of 1. If either of them is 0, the

total input will be less than 2 and the output is 0. So, this models an AND, and the output is $X \text{ AND } Y$.

In the figure to the right, whenever the input X is 0, the weighted input is 0 which matches the threshold, so the unit will output a 1. But if the input X is 1, the total weighted input is less than the threshold and the unit will output 0. So, this perceptron models, the Boolean relation, NOT X . In the figure to the bottom, the weighted sum of inputs becomes 1 and matches the threshold, if either X or Y is 1. So, this models the relation $X \text{ OR } Y$. But Minsky and Papert showed that there are some Boolean functions that a perceptron simply cannot compute for any setting of its weights. For example, there is no setting of the weights for which the unit can compute $X \text{ XOR } Y$. In fact, most Boolean functions cannot be computed by just a single perceptron.

So, how do we model such functions with perceptrons? To understand this, let us go back to the brain. A brain is not just one single neuron — it is in fact a network of neurons. And similarly, for our computational model too, although individual perceptrons are rather weak computational elements, you can greatly extend what you can do with them, once you begin networking many of them. If you just connect up three perceptrons, you can compute an XOR as shown in this figure. Here, we have connected three perceptrons in a layered manner. We have two perceptrons in the first layer, which directly operate on the inputs. Then we have a third perceptron, which operates on the outputs of the first two perceptrons. The first perceptron here computes $X \text{ OR } Y$. The second perceptron computes NOT X or NOT Y . The final perceptron ANDs the output of the first two perceptrons, to give you $X \text{ XOR } Y$.

Some terminology here — This first layer of perceptrons is what we call a hidden layer, because you don't actually need to see the outputs of these units — they can be hidden. You only need to see the output of this final perceptron, the output perceptron. This entire, layered network of perceptrons is what we call a multilayer perceptron. Once you realize that you can begin connecting up perceptrons into a network to compute more complex Boolean functions, you realize that you can compute arbitrarily complex Boolean functions simply by connecting up perceptrons in the right way.

For example, this network here operates on four inputs X , Y , Z and A , and computes this complex function of the four inputs. A network of perceptrons can indeed be a universal model for Boolean functions — you can compute any Boolean function with an appropriately structured network of perceptrons. So, the story so far is this: Neural networks are connectionist models that model the structure of the brain. The basic unit is Rosenblatt's perceptron which is a simple threshold gate model of the biological neuron. Perceptrons can model Boolean gates. Multilayer perceptrons can model arbitrarily complex Boolean functions.

Video 1.3: Multilayer Perceptrons as Functions (04:44)

So, our story so far: Multilayer perceptrons (MLP) are connectionist computational models. Individual perceptrons are computational equivalents of a neuron. The MLP is a network of

perceptrons. MLPs can model any Boolean function — they are universal Boolean functions. MLPs can also model any decision boundaries — they are universal classifiers. So far, we've seen networks that produce Boolean, or possibly even categorical outputs.

But what about continuous-valued outputs? Can we construct networks to model continuous-valued functions like the one in this figure? To see how this could be done, consider a function of a scalar input. You could have two perceptrons, one of which outputs a 1 when the input exceeds the threshold T_1 , and a second one which outputs a one when the input exceeds a threshold T_2 . We'll assume without loss of generality that T_1 is less than T_2 . You add the outputs of the two perceptrons with weights 1 and -1 respectively. Now consider the overall response of this net. When the input x is less than T_1 , both perceptrons output 0, so the overall output of the net is 0. When the input is between T_1 and T_2 , the first perceptron fires, but the second one does not, so overall output of the network is 1. When the input is greater than T_2 , both perceptrons fire. But because their outputs are added with weights 1 and -1, they will cancel each other out and the overall output becomes 0.

So, this net produces an output of 1 when the input is between T_1 and T_2 , and 0 otherwise. Now I can easily extend this to model any scalar function of a scalar input. Say I want a network that approximately gives me the function represented by this blue curve. I partition the axis into ranges and use one subnet of 2 neurons to produce a square pulse of height 1 in that range, scale the outputs of the individual subnets appropriately by weighting their outputs correctly, and add them all up, to get the approximation shown here. I can make the approximation arbitrarily precise by making these ranges narrower. In other words, a multilayer perceptron (MLP) can approximate any scalar function of scalar inputs to arbitrary precision.

We can generalize this principle to functions of multivariate or vector inputs. And so, adding to our story — MLPs are not only universal Boolean functions, and universal classifiers, they can also model any continuous-valued function. So, connecting back to our original question of how neural networks perform all the fancy tasks that we saw that they could do: From an AI perspective, a neural network is just a function. Given one or more inputs, it computes the function to predict one or more outputs. In all of these tasks, which we saw early on in the lecture, these boxes are just functions, which take in an input and produce an output. For example, the voice recognition function takes in voice and produces a transcription. The image captioning function takes in an image and generates a caption. The game-playing function takes in a game state and outputs the best next move. And when we do it with neural nets, all of these functions are modeled or approximated by neural networks.

So, let's add one more line to our story so far: Multilayer perceptrons (MLPs) are universal function approximators. The various AI tasks that we saw are just functions, and so an MLP can be built to compute all of these functions.

Video 1.4: Multilayer Perceptron Layers (04:36)

We've described what an MLP is. Before we proceed, let's address one more question. The course is titled deep learning, referring to deep neural networks. So, let's take a few minutes to define what it means for an MLP to be deep. We have seen that neural networks are composed of networks of computational models of the biological neuron called perceptrons. Often, we will just call the perceptrons neurons, so if you hear me say neuron, remember that I'm referring to one of the computational units in the network.

A multilayer perceptron, or an MLP, is a network of such perceptrons, where perceptrons receive inputs from other perceptrons. We will see the formal definition of a layer later. When these networks have many such layers, they're said to be deep networks. So, what is the formal notion of depth in this context? For that matter, what do we mean by a layer? In any directed graph with sources and sinks, the depth of the graph is simply the length of the path — that is the number of edges — on the longest path from a source to a sink. By source, we mean a node that has only outgoing edges.

A sink is a node with only incoming edges. Both the graphs here have one source and one sink. So, for example in the graph to the left, there are two paths from the source to the sink. One is length 2, another is length 1. The depth of this graph is 2. In the graph to the right, there are paths of lengths 1, 2, and 3 from the source to the sink. The longest path is length 3, so the depth of this graph is 3. We apply the same principle in defining neural networks.

The computation in a neural network is generally directed, from input to output. The depth of a network is the length of the longest path, that is, the maximum number of perceptrons, or neurons, from input to output. So, in the first figure the depth of the network is 3 — since the longest path from the input to output is 3. In the figures to the right, each box represents a layer of neurons. So, you also have paths of different lengths from input to output. In the upper right figure, the depth of the net is 2. In the lower right figure, it is 4. A deep network is a network whose depth is greater than 2, it is at least 3. So, on this slide the net on the top right is not deep, but the other two are. I keep speaking of layers — what is a layer in a network? A layer is the set of neurons that all have the same depth from the input.

The depth of the layer is the depth of all the neurons in it from the input. For example, in this figure, all the green neurons have a depth 2, so they form a layer, which we call the second layer. Note that there is also a direct path of length 1 from the input to these neurons, but they're not part of the first layer. The first layer is this set of red neurons at depth 1, the yellow neurons are the third layer, and the blue neurons are the fourth layer.

So, our story so far is this: Neural networks are universal approximators. They can be constructed to approximate any function. They are generally organized as deep networks with many layers. The depth of a neuron is the length of the longest path from the input to the neuron. A layer is the set of neurons at the same depth. A deep network is one with at least 3 layers.

Video 1.5: Multilayer Perceptron Parameters (04:43)

So, we know that neural nets are capable of modeling any function at all, but how do we make the model a specific function of interest? That will be the second half of this and the next couple of modules. Here's what we know so far. Neural networks are universal approximators. They can model any Boolean, classification, or regression function. We also saw that complex task such as speech recognition, image captioning, or playing games are functions. They take in an input, like the voice signal, the image, or the current game state, and compute an output, like the transcription for the recording, the image caption, or the next game state. So, they can be modeled by a neural net.

Let's now consider the mechanics of it. The network, as a function, operates on numbers — it takes in numbers and outputs numbers. So, for it to perform all the tasks we saw, all inputs and outputs must somehow be represented as numbers. So, our first question is — how do we represent the input and output of the network as numbers? Second, we know that we can construct a network to compute the functions that perform all of these tasks, but how do we actually compose it? We'll deal with this first question later and focus for now on this second question.

Let's start with the basic units of our network: the perceptron. The perceptron operates on real-valued inputs. It first computes an affine function of the inputs, which is a weighted sum of all the inputs plus a bias. This affine function, which we will generally represent as Z in our lectures, is then put through an activation function. We've used threshold activations in our explanation so far, but in reality, it can be anything. The parameters of the perceptron, which determines how it behaves, are its weights and bias.

Redrawing the neuron instead of having an explicit bias, we can also think of it as having one more input component whose value is permanently fixed to 1. The bias term simply becomes the weight associated with this extra fixed input. Often in the slides, we won't explicitly show the bias. In all those cases, you must assume that every perceptron actually has this additional fixed input, that implicitly accounts for the bias. The MLP is a network of perceptrons.

We will assume a feed-forward network, which means that computations are performed in a strictly directed way, proceeding unidirectionally from input to output — left to right in this figure. When we process an input, each neuron is computed exactly once. A major part of designing such a network is the architecture of the net: how many layers does it have, how many neurons are there in each layer, how they are connected, etc. For now, though, we will assume that architecture of the network is given to us and is capable of representing the needed function. So, the entire network is a function. It takes an input and produces an output.

The parameters of the function are the weights and biases of the neurons in the network — We can represent the function as $f(X;W)$, where X is the input, and W are the network parameters. To model any specific function with a network, we must set these parameters appropriately for it. When we speak of learning the network, we're speaking of finding the weights and biases of this network such that the network computes the desired function.

Video 1.6: How to Learn a Network (07:35)

So, here's what we know: given any function $g(X)$, we can construct an MLP that computes it. But then the question is, how do we construct it? The MLP is a function $f(X;W)$, where W are the parameters — weights and biases of the network. So, the question actually is — how do we set W such that $f(X;W)$ computes $g(X)$?

So, here's the setup: We are given $g(X)$ — shown by the dotted blue curve in this figure. Here, the X axis represents the input. The network itself is a function $f(X)$ with parameter W . For any given setting of W , the network will compute some function, not necessarily equal to $g(X)$. That is shown by the dotted red line here. The shaded area represents the error between what the network computes and the target function $g(X)$. So, to learn the function, we learn W to minimize this area.

First, we must quantify this area. To quantify this area, we will define a divergence function $\text{div}()$ that quantifies the discrepancy between the actual output of the network $f(X;W)$ and the desired output $g(X)$ at any X , which is illustrated by the line in this figure. The divergence function has the property that it goes to 0 when $f()$ is exactly equal to $g()$ and is positive when the two are not the same. For our purpose, we also want it to be differentiable with respect to $f(X)$. This entire area is the integral of the divergence function, computed over all valid values of the input. This quantifies the error between the actual output of the network, and the target function $g(X)$. And so, we want to compute the W to minimize this integral. Because the network is only guaranteed to be an approximation, we can be more conservative.

The probability distribution of the input X , shown by the blue curve here, will rarely be uniform. Not all values of X are equally common — some values are very common; others are rarely or never seen. We want to make sure we have low error for more probable values of X , whereas we don't even care what the error is for impossible values of X . So, we'll modify our error metric — we will now multiply the divergence at each X by the probability of X , within this integral to give more importance to more likely values of input X . This is simply the expected value of the divergence between $f(X)$ and $g(X)$ for that specific setting of network parameters W . And so, to find the best network to model a function $g()$, we must find the parameters W that minimize the expected divergence between the network output $f(X)$ and the target $g(X)$. This is the actual objective of training.

The problem is, in order to compute this integral, the function $g(X)$ must be fully specified everywhere. In practice, when we are trying to build a network for a task to compute a function for any task, like classifying patterns, we will generally not have a full specification of the function to begin with. So instead, we will sample the function. Basically, we will collect a set of samples, X and $g(X)$, where at each sample we know both the input X and the function output $g(X)$. It is very easy to collect such input-output pairs. You just literally collect data from the real world, like images and their class labels or speech recordings and their transcriptions. These samples are our training samples.

We must now learn the entire function from these training samples. At each of these training points X_i , the network will have an output $f(X_i)$. The desired output will be some d_i . The discrepancy between the two, shown by these red lines here, will be the divergence between d_i and $f(X_i)$. We compute the average of the divergences at the training points — the average of the values represented by these red lines — and use that as a substitute of the expected divergence, which is the shaded area. We will minimize this average training set divergence, and hope that it minimizes the entire shaded area.

And so, here's the setup for learning the network to model any function. We collect a set of training samples, each of which includes an input and the desired output. We define a divergence between the actual and desired output of the net. We define a loss function that is the average divergence computed at all the training samples. And we learn the network parameters to minimize this average divergence. The terminology that is normally used for this term, which quantifies the network's error, is loss. It is the average divergence computed at the training instances and is only an approximation to what we really want to minimize, which is the expected divergence.

Since the inputs and desired outputs are given, the only variable we can change to change the loss is W — the network parameters. For a given training set, the loss is a function only of W . So, here's our final problem statement. We are given a training collection of input-output pairs for the target function. We compute the loss, which is the average divergence of the network from the target outputs at these training points. The loss is a function of W , so we minimize it with respect to W . This is a problem of function minimization — it's an optimization problem.

So, the story so far is this: We learn networks by fitting them to training instances drawn from a target function. We define a differentiable divergence between the output of the network and the desired output for the training instance, and a loss, which is the average divergence over all training instances. We optimize network parameters to minimize this error. This is an instance of function minimization

Video 1.7: A Note About Derivatives (06:51)

First, a brief tutorial on derivatives. The derivative of a function at any point tells us how much a minute increment to the argument of the function will increment the value of the function. For any smooth continuous function $y = f(x)$, the derivative of the function at any x is a multiplicative factor by which you have to multiply a small increment in x , to obtain the corresponding increment of y . So, at any location x , if α is a derivative at that location, then for a small perturbation Δx of x , the resulting change Δy of y is given by $\Delta y = \alpha \Delta x$. We should actually write it as $\alpha(x)$, since this is a function of x .

For a scalar function of a scalar variable, y and x are scalar. The derivative α is a scalar value by which we must multiply an increment dx of x to get the corresponding increment $dy = \alpha dx$. In this scalar case, it's reasonable to represent the derivative as dy by dx but the notation $f'(x)$ is more reasonable, as it shows the explicit dependence of the derivative on the

current location x . The derivative at any point is actually the rate of change of the function at that point or the slope of the function. If the function is steep, this will be large in magnitude, if the function is shallow, it will be small in magnitude. If increasing x increases $f(x)$, like in these regions marked with the plusses, then the derivative will be positive. Increasing x decreases $f(x)$, like in the regions marked with the minuses, the derivative will be negative. And if the function is locally just flat, not increasing or decreasing, like at the locations touching the horizontal lines, the derivative will be 0. When the argument to the function is a vector, then x is a vector and Δx is also a vector.

Once again, the derivative is this multiplier α that you multiply Δx by to compute the increment Δy . If x is a column vector, Δx is also a column vector. Δy is a scalar. So, for this relation to hold, it must be a row vector given by this expression here. So, Δy is given by these two equations, where Δx_1 , Δx_2 , etc., are the components of Δx . Each of these individual α components, α_i are partial derivatives of the function with respect to one of the components of x . It tells you how much y increments when only that x_i is incremented and all the other dx terms are 0. This is often represented as partial y over partial x_i . And you naturally also get the overall increment in y as shown by these three equations. So, we can write the derivative equation as the expression shown here, where instead of α , we are representing the derivative by this inverted triangle, nabla. This notation here means that we are computing the derivative of y with respect to the argument x which is in the subscript. We will be using this symbol for vector and matrix derivatives going forward. This derivative, of course, is the row vector composed of the partial derivatives of y with respect to the components of x as shown here.

You may be more familiar with the term gradient to describe the derivative of scalar functions with respect to vector arguments. The gradient is a column vector composed of the partial derivatives of the function with respect to the components of x . This is actually the transpose of the derivative. The property of the gradient is that it is the direction of fastest increase in $f(x)$. At any point x , you can take a step in many directions and each step will result in a change of y . The gradient is the specific direction where y increases fastest. So, moving in the direction of the gradient will result in the fastest increase in the function $f()$. The length of the gradient gives you the rate of increase of the function $f()$ in this direction. Conversely, if you walk in the exact opposite direction to the gradient — in the direction of the negative of the gradient, that is the direction in which the function $f()$ decreases fastest. At local minima, the gradient is 0. It doesn't matter which direction you make an incremental shift in starting from here, the function will remain flat.

We can also compute the second derivative of a function of a vector, with respect to its argument. This is called the Hessian. It will be a matrix, where the ij -th component is the partial derivative of f with respect to x_i and x_j , the i th and j th input components. I'm defining it here because we'll encounter it again shortly. The eigenvalues of the Hessian matrix defined the quadratic curvature of the function f along its principal directions. So, at a strict minimum, the eigenvalues will all be positive because the function will be a minimum along every direction. At a strict maximum, the eigenvalues will all be negative. At other locations, there will be a mix

of positive and negative values and even 0. We'll stop here and return to our topic of function optimization next.

Video 1.8: Gradient Descent and Learning Rate (05:33)

Before we close this module, let's look at the actual behavior of gradient descent. We've learned so far that neural nets can be trained by minimizing a loss, that quantifies the average error it makes on training points. The loss is a function of network parameters. Complicated functions like the loss function can be minimized through gradient descent. But will gradient descent actually find the minimum?

The fact is, complicated functions like the loss function can have many local minima, or other points where the gradient is 0, such as inflection or saddle points. Gradient descent can get stuck at these points, whereas we want it to arrive at the global minimum in the loss. We have assumed that gradient descent steps will eventually converge to a location where the gradient is 0. But in reality, this may not happen. The iterations may end up jittering around the minimum, or even diverging.

So, what are the conditions for convergence? To understand, let's look at the gradient descent update rule again. To minimize some function $L(w)$ with respect to its argument w , the gradient descent update rule is given here: The $(k+1)$ th estimate equals $w^{(k)}$ minus η times the derivative of the function at $w^{(k)}$. As we mentioned earlier, η is the learning rate that determines the actual size of your step. For any function, at any current estimate, there is an optimal learning rate, η_{opt} , such that if we use that learning rate, the gradient descent update will arrive at the minimum in just one step, as shown in the first figure. For quadratic functions, the optimal η can be shown to be the inverse of the second derivative.

There's also a divergent learning rate, η_{div} , where if your learning rate is equal to or greater than this value, the gradient descent will actually diverge and take you further from the minimum than you currently are. For quadratic functions, this is actually twice the optimal learning rate. In general, the farther the learning rate is from the optimal, the more time gradient descent will take to arrive at the minimum, if it ever even arrives at the minimum. Ideally, the learning rate should be close to the optimal learning rate and less than the divergent rate in any case. But all of this was based on trying to ensure that the learning rate was not so large as to cause divergence within a convex region. How critical is this? The loss function is generally very complex with many local minima.

The initial estimate can be anywhere. If the learning rate is small enough to prevent divergence, it will only push you to the bottom of the local minimum below it, whereas the global minimum is somewhere else. In this case what you really want is a learning rate that is divergent, so that the estimate actually escapes this local minimum. However, always keeping the learning rate divergent will also prevent the algorithm from ever converging to any solution.

What we want is to start with a learning rate that is large enough to cause the algorithm to diverge and escape any local minima, but then slowly reduce the learning rate. In the beginning, it will keep jumping out of local minima, but it will eventually find a bowl that's big enough that the estimate cannot escape it. That bowl will typically be deeper, and closer to being a global minimum than the location of the initial estimate. Reducing the learning rate with iterations is called learning rate decay. A number of different methods have been proposed. Linear decay is where the initial learning rate η_0 shrinks in inverse proportion to the iterations.

In quadratic decay, it decays in inverse proportion to the square of the iteration number. Or you can have exponential decay, where the learning rate decays exponentially with iterations. In neural networks, a common approach is to begin with a fixed learning rate η until the loss, or performance on a held-out data set, stagnates. Then shrink the learning rate by a factor, typically 0.1, and continue till the loss stagnates, then shrink it again.

So, the story so far is that: Gradient descent may not find the right answer. The loss surface has many non-optimal fixed points and gradient descent may find any of these. We also learned that convergence depends on the learning rate. Divergence in initial steps is not a bad thing, but continuous divergence is not good. Ideally, we'd start with a large learning rate and shrink it with iterations. In the next module, we'll examine gradient descent to actually train a network.